

ANKARA ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

Ağ Tabanlı Teknolojiler ve Uygulamaları

FİNAL PROJESİ RAPORU

SOSYAL İÇERİK & QUIZ PLATFORMU

Hazırlayan: Elif Özçelik- 22290116

Ocak 2026

Video Linki:

https://drive.google.com/drive/folders/1eLbGI8x_wDEdTDaIK4PM2SPmXPPSoQwI?usp=drive_link

Github Linki: https://github.com/Elifozclik/Quiz_Platform-ASP.NET

1.0 Projeye Genel Bakış

1.1 Giriş

Bu doküman, kullanıcıların etkileşimli kişilik testleri ve quizler oluşturup paylaşabildiği bir web uygulaması olan **Sosyal İçerik & Quiz Platformu**'nun teknik mimarisini, bileşenlerini ve temel mekanizmalarını detaylandırmaktadır. Projenin temel amacı, son kullanıcılara eğlenceli ve angaje edici bir deneyim sunarken, arka planda modern web teknolojilerini etkin bir şekilde uygulamaktır. Bu belge, projeyi devralacak veya projeye yeni katılacak geliştiriciler için kritik bir başlangıç kaynağı olarak tasarlanmıştır. Sistemde yer alan sunum, API ve veri katmanlarının bütüncül bir resmini sunarak, geliştiricilerin kod tabanına hızla adapte olmalarını ve verimli bir şekilde geliştirme yapmaya başlamalarını sağlamayı hedefler.

1.2 Temel Özellikler

Platformun işlevsel ve teknik kabiliyetleri aşağıdaki ana başlıklar altında toplanmıştır:

- **Kullanıcı Yönetimi:** Sistem, kullanıcıların e-posta ve şifre ile kayıt olmasını, giriş yapmasını ve profil resmi yüklemesini destekler. Ayrıca, kullanıcıların hesaplarını geçici olarak devre dışı bırakma ve daha sonra yeniden aktifleştirme gibi esnek hesap yönetimi özellikleri sunar.
- **İçerik Yönetimi (Quiz):** Yöneticiler, görsel destekli (kapak resmi, soru ve şık görselleri) quizler ve kişilik testleri oluşturabilir. Kişilik testleri, her bir şık için farklı kişilik tiplerine puan atayan esnek bir JSON tabanlı puanlama algoritması kullanır. Bir quiz silindiğinde, veritabanı seviyesinde cascade delete mekanizması tetiklenerek quiz ile ilişkili tüm alt kayıtlar (sorular, şıklar, sonuçlar vb.) otomatik olarak temizlenir.
- **Yönetim Paneli (Admin):** Yöneticiler için tasarlanmış kapsamlı bir yönetim paneli mevcuttur. Bu panel üzerinden kullanıcı hesapları, oluşturulan tüm quizler ve kullanıcıların hesap silme talepleri merkezi olarak yönetilebilir.
- **API ve Entegrasyon:** Uygulamanın çekirdeğinde, toplam 12 adet endpoint içeren bir RESTful API katmanı bulunmaktadır. Bu API, sunum katmanının veri işlemleri için kullandığı ana arayüzdür ve /swagger adresi üzerinden erişilebilen detaylı bir dokümantasyona sahiptir.
- **Güvenlik ve Kimlik Doğrulama:** Kimlik doğrulama işlemleri, endüstri standardı olan Session tabanlı (Session-based authentication) bir mekanizma üzerine kurulmuştur. Bu yapı, kullanıcı oturumlarını güvenli ve durum bilgisi korunacak şekilde yönetir.
- **Kullanıcı Arayüzü:** Kullanıcı arayüzü, Bootstrap 5 framework'ü kullanılarak geliştirilmiştir. Bu sayede platform, tüm cihazlarda (mobil, tablet, masaüstü) tutarlı ve duyarlı (responsive) bir kullanıcı deneyimi sunar.

Bu temel özellikler, bir sonraki bölümde detaylandırılacak olan sistem mimarisi tarafından desteklenmektedir.

2.0 Sistem Mimarisi ve Teknoloji Yığını

2.1 Mimari Yapı Analizi

Projenin mimari yapısı, ASP.NET Web Forms ve Web API teknolojilerini bir araya getiren hibrit bir yaklaşım üzerine kurulmuştur. Bu stratejik seçim, Web Forms'un hızlı arayüz geliştirme yetenekleri ile Web API'nin sunduğu esnek ve modern hizmet katmanı avantajlarını birleştirmeyi amaçlamıştır.

Uygulama, sorumlulukların net bir şekilde ayrıldığı klasik **üç katmanlı (Three-Tier)** mimari modelini benimser. Bu model; Sunum Katmanı, API Katmanı ve Veri Erişim Katmanı olarak ayrılmıştır. Katmanlar arasındaki bu ayrım, sistemin modülerliğini artırır, bakımını kolaylaştırır ve gelecekteki geliştirmeler için esnek bir zemin hazırlar.

Uygulamanın veri akışı aşağıdaki şema ile özetlenebilir:

Sunum Katmanı (ASPX) <-> API Katmanı (Web API) <-> Veri Erişim Katmanı (DbHelper) <-> Veritabanı (MySQL)

2.2 Teknoloji Yığını

Platformu oluşturan temel teknolojiler, backend ve frontend olarak iki ana kategoride aşağıda detaylandırılmıştır.

Backend Teknolojileri

Teknoloji	Açıklama
ASP.NET 4.7.2	Projenin ana omurgasını oluşturan, Web Forms tabanlı sunucu tarafı uygulama çatısıdır.
Web API 2.0	Uygulamanın veri işlemlerini yürüten ve 12 endpoint içeren RESTful API katmanını sağlar.
C#	Tüm backend mantığının geliştirildiği ana programlama dilidir.
MySQL 8.0	Uygulamanın tüm verilerini depolayan ilişkisel veritabanı yönetim sistemidir.
ADO.NET	Veritabanı ile güvenli iletişimi sağlar. Parameterized query'ler ile SQL Injection zafiyetine karşı koruma sunar.
Swagger	RESTful API endpoint'lerinin otomatik olarak dokümanite edilmesini ve test edilmesini sağlar.

Frontend Teknolojileri

Teknoloji	Açıklama
Bootstrap 5	Duyarlı (responsive) ve modern bir kullanıcı arayüzü oluşturmak için kullanılan CSS framework'üdür.
JavaScript	Tarayıcı tarafında dinamik ve interaktif kullanıcı deneyimleri yaratmak için kullanılır.
jQuery	DOM manipülasyonu, olay yönetimi ve AJAX istekleri gibi işlemleri basitleştiren bir JavaScript kütüphanesidir.
CKEditor	Quiz açıklamaları gibi alanlarda zengin metin düzenleyici (rich text editor) işlevselliği sunar.

Bu teknoloji yığını, uygulamanın temelini oluşturan veritabanı şemasının detaylı inceleneceği bir sonraki bölüme zemin hazırlamaktadır.

3.0 Veritabanı Şeması ve Veri Modeli

3.1 Veritabanına Giriş

Veritabanı tasarımı, platformun veri bütünlüğünü ve performansını doğrudan etkileyen en kritik bileşenlerden biridir. Bu proje kapsamında, ilişkisel veri modelinin gücünden faydalanmak üzere **MySQL 8.0** tercih edilmiştir. Toplam 8 adet tablodan oluşan şema, uygulamanın tüm varlıklarını (kullanıcılar, quizler, sorular vb.) ve aralarındaki ilişkileri mantıksal bir yapıda saklar. Tablolar arasındaki referans bütünlüğü (referential integrity), Foreign Key kısıtlamaları kullanılarak veritabanı seviyesinde garanti altına alınmıştır.

3.2 Tablo Yapıları

Veritabanını oluşturan temel tablolar ve sorumlulukları aşağıda özetlenmiştir.

Tablo Adı	Açıklama
users	Kullanıcıların temel bilgilerini içerir. Kritik alanlar: şifre hash (SHA256), rol (admin/user), profil resmi yolu ve IsActive durumu.
quizzes	Oluşturulan quizlerin ana bilgilerini tutar. Başlık, açıklama, kapak görseli ve yayın durumu (Published/Draft) gibi verileri içerir.
questions	Quizlere ait soruları barındırır. Soru metni, görseli ve sırası gibi bilgileri saklar.
options	Sorulara ait şıkları içerir. Şık metni, görseli ve en önemlisi, kişilik testleri için puanlama mantığını içeren PersonalityScores JSON verisini tutar.
results	Kişilik testlerinin olası sonuç tiplerini tanımlar (örn: "Aslan"). Başlık, açıklama, kişilik tipi kodunu ve skor tabanlı quizler için MinScore/MaxScore aralıklarını içerir.
submissions	Kullanıcıların bir quizi tamamladığında oluşturulan ana kayıt. Hangi kullanıcının hangi quizi çözdüğünü, aldığı sonucu (ResultId), hesaplanan toplam skoru (TotalScore) ve anonim çözümler için kullanılan anahtarı (AnonymousKey) saklar.
submissionanswers	Bir submission kaydına ait detaylı cevapları tutar. Kullanıcının her soru için hangi şıkkı seçtiğini kaydeder.
deleterequests	Kullanıcıların hesap silme taleplerini yönetir. Talebin durumu (pending, approved, rejected) bu tabloda saklanır.

3.3 Tablo İlişkileri ve Cascade Delete Mekanizması

Tablolar arasındaki ilişkiler, Foreign Key'ler aracılığıyla kurulmuştur. Bu yapı, veri tutarlılığını sağlar ve CASCADE DELETE mekanizmasının etkin bir şekilde çalışmasına olanak tanır.

- users.Id → quizzes.CreatedBy
- users.Id → submissions.UserId
- users.Id → deleterequests.UserId
- quizzes.Id → questions.QuizId
- quizzes.Id → results.QuizId

- quizzes.Id → submissions.QuizId
- questions.Id → options.QuestionId
- results.Id → submissions.ResultId
- submissions.Id → submissionanswers.SubmissionId

CASCADE DELETE Mekanizması: Bu mekanizma, veri bütünlüğünü korumak için kritik bir rol oynar. Örneğin, bir yönetici quizzes tablosundan bir kaydı sildiğinde, bu quiz ile Foreign Key üzerinden ilişkili olan tüm questions, options, results, submissions ve submissionanswers kayıtları veritabanı tarafından otomatik olarak ve güvenli bir şekilde silinir. Bu özellik, uygulama kodunda karmaşık silme mantıkları yazma ihtiyacını ortadan kaldırır ve veritabanında "yetim" kayıtlar kalmasını engeller.

Bu veri yapısı üzerine inşa edilen uygulamanın güvenlik ve kimlik doğrulama gibi temel mekanizmaları sonraki bölümde incelenecektir.

4.0 Temel Mekanizmalar ve Güvenlik

4.1 Güvenlik Önlemleri

Uygulamanın güvenliği, potansiyel tehditleri proaktif olarak bertaraf etmek üzere tasarlanmış çok katmanlı bir yaklaşımla sağlanmıştır. Stratejik olarak seçilen bu önlemler, hem kullanıcı verilerini hem de sistem bütünlüğünü korumayı hedefler.

4.1.1 Parola Güvenliği: SHA256 Hashing

Kullanıcı parolaları, veritabanında asla düz metin (plaintext) olarak saklanmaz. Bunun yerine, endüstri standardı olan tek yönlü bir kriptografik özet fonksiyonu olan SHA256 algoritması kullanılır. Bu algoritma, girilen parolayı geri döndürülemez bir "hash" değerine dönüştürür. Bu sayede, veritabanına yetkisiz erişim sağlansa bile kullanıcı parolaları ele geçirilemez.

```
using System.Security.Cryptography;
```

```
public static string HashPassword(string password)
```

```
{
```

```
    using (SHA256 sha256 = SHA256.Create())
```

```
    {
```

```
        byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
```

```
        StringBuilder builder = new StringBuilder();
```

```
        for (int i = 0; i < bytes.Length; i++)
```

```
        {
```

```
            builder.Append(bytes[i].ToString("x2"));
```

```
        }
```

```
        return builder.ToString();
```

```
    }
```

```
}
```

4.1.2 SQL Injection Koruması

SQL Injection, saldırganın uygulama sorgularına müdahale ederek veritabanı üzerinde yetkisiz işlemler yapmasına olanak tanıyan yaygın bir zafiyettir. Bu projede, veritabanına gönderilen tüm sorgularda Parameterized Query (parametrelili sorgu) tekniği kullanılarak bu risk tamamen ortadan kaldırılmıştır. Bu teknikte, kullanıcıdan gelen veriler sorgu metnine doğrudan eklenmek yerine, sorgu şablonuna güvenli bir şekilde parametre olarak bağlanır.

YANLIŞ (Güvenlik Açığı İçerir):

```
string query = "SELECT * FROM users WHERE Email = '" + email + "'";
```

DOĞRU (Güvenli Yöntem):

```
string query = "SELECT * FROM users WHERE Email = @Email";
```

```
MySQLParameter[] parameters = {  
    new MySQLParameter("@Email", email)  
};
```

```
DataTable result = DbHelper.ExecuteQuery(query, parameters);
```

4.1.3 Cross-Site Scripting (XSS) Koruması

Cross-Site Scripting (XSS), saldırganın başka kullanıcıların tarayıcısında zararlı betikler (<script>) çalıştırmasına olanak tanıyan bir zafiyettir. Bu genellikle, kullanıcıdan alınan verilerin (örneğin, quiz açıklaması) filtrelenmeden doğrudan sayfaya yazdırılmasıyla ortaya çıkar. Projede, kullanıcı tarafından girilen ve HTML içerebilen tüm veriler, veritabanından okunup sayfaya yazdırılmadan önce bir HTML temizleme (sanitization) fonksiyonundan geçirilir. Bu fonksiyon, <script> ve <iframe> gibi tehlikeli etiketleri içerikten temizler.

```
public static string SanitizeHtml(string html)  
{  
    if (string.IsNullOrEmpty(html)) return "";  
  
    // Tehlikeli tagları kaldır  
    html = Regex.Replace(html, @"<script[^>]*>.*?</script>", "",  
        RegexOptions.IgnoreCase | RegexOptions.Singleline);  
    html = Regex.Replace(html, @"<iframe[^>]*>.*?</iframe>", "",  
        RegexOptions.IgnoreCase | RegexOptions.Singleline);  
  
    return html;  
}
```

4.1.4 Dosya Yükleme Güvenliği

Güvenli olmayan dosya yükleme mekanizmaları, sunucuda yetkisiz kod çalıştırılması gibi ciddi güvenlik riskleri oluşturabilir. Bu platformda, profil ve quiz görsellerinin yüklenmesi sırasında aşağıdaki katı validasyon adımları uygulanır:

- **Uzantı Kontrolü:** Sadece izin verilen dosya uzantıları (.jpg, .png vb.) kabul edilir (whitelist yaklaşımı).
- **Dosya Boyutu Kontrolü:** Maksimum dosya boyutu 5MB ile sınırlandırılmıştır.
- **Görüntü Boyutları Kontrolü:** Yüklenen görsellerin minimum ve maksimum piksel boyutları (örn: 200x200 - 2000x2000) kontrol edilir.
- **Path Traversal Koruması:** Yüklenen dosyanın adı, GUID kullanılarak oluşturulan benzersiz bir isimle değiştirilir. Bu, saldırganların dosya adını manipüle ederek (../..) sunucu dosya sisteminde gezinmesini engeller.

4.2 Kimlik Doğrulama ve Session Yönetimi

Uygulamanın kimlik doğrulama ve yetkilendirme altyapısı, ASP.NET Session mekanizması üzerine kurulmuştur. Kullanıcı başarılı bir şekilde giriş yaptığında, oturum süresince geçerli olacak temel bilgiler sunucu tarafında Session nesnesinde saklanır.

Giriş sonrası set edilen temel Session değişkenleri şunlardır:

- Session["UserId"]: Oturum açan kullanıcının veritabanındaki benzersiz kimliği.
- Session["UserType"]: Kullanıcının rolünü belirtir (admin veya user). Yetkilendirme kontrollerinde kullanılır.
- Session["UserName"]: Arayüzde gösterilmek üzere kullanıcının adı.
- Session["UserEmail"]: Kullanıcının e-posta adresi.

Yönetim paneli gibi yetki gerektiren tüm sayfalarda, Page_Load event'i içinde bu Session değişkenleri kontrol edilir. Eğer oturum açılmamışsa veya kullanıcının rolü yetersizse, kullanıcı otomatik olarak giriş sayfasına yönlendirilir.

// Admin.Master sayfasındaki yetkilendirme kontrolü

```
protected void Page_Load(object sender, EventArgs e)
```

```
{  
    if (Session["UserType"]?.ToString() != "admin")  
    {  
        Response.Redirect("~/Pages/User/Login.aspx");  
    }  
}
```

Bu temel mekanizmalar, bir sonraki bölümde incelenecek olan uygulama sayfalarında somut işlemlere dönüşmektedir.

5.0 Uygulama Akışı ve Sayfa Analizleri

5.1 Sayfa Mimarisi: Master Pages

Platformda tutarlı bir kullanıcı arayüzü sağlamak ve kod tekrarını en aza indirmek amacıyla Master Page yapısı etkin bir şekilde kullanılmaktadır. Bu yapı, tüm sayfalarda ortak olan navigasyon menüsü, alt bilgi (footer) gibi bileşenlerin tek bir merkezden yönetilmesini sağlar.

5.1.1 Site.Master (Genel ve Kullanıcı Sayfaları)

Site.Master, hem kimlik doğrulaması gerektirmeyen genel (public) sayfalar hem de giriş yapmış kullanıcılara özel sayfalar için ortak şablon görevi görür. En önemli özelliği, kullanıcının oturum durumuna (Session) göre navigasyon menüsünü dinamik olarak oluşturmasıdır. Eğer kullanıcı giriş yapmışsa "Dashboard", "Profil" ve "Çıkış" linkleri gösterilirken, misafir kullanıcılar için "Giriş" ve "Kayıt Ol" linkleri görüntülenir.

```
// Site.Master'daki navigasyon mantığı
```

```
if (Session["UserId"] != null)
{
    // Giriş yapmış kullanıcı
    loginLink.Visible = false;
    registerLink.Visible = false;
    userMenu.Visible = true;
    userName.Text = Session["UserName"].ToString();
}
else
{
    // Misafir
    userMenu.Visible = false;
}
```

5.1.2 Admin.Master (Yönetim Paneli)

Admin.Master, yalnızca yönetim paneli sayfaları için tasarlanmış özel bir şablondur. Bu şablonun en kritik sorumluluğu, her sayfa yüklendiğinde (Page_Load event'i) yetkilendirme kontrolü yapmasıdır. Session["UserType"] değişkenini kontrol ederek, rolü admin olmayan tüm kullanıcıları (hem user rolündekileri hem de misafirleri) otomatik olarak giriş sayfasına yönlendirir ve yönetim paneline yetkisiz erişimi engeller.

```
// Admin.Master'daki yetkilendirme kontrolü
```

```
protected void Page_Load(object sender, EventArgs e)
{
    if (Session["UserType"]?.ToString() != "admin")
```



```
{  
    Response.Redirect("~/Pages/User/Login.aspx");  
}  
}
```

5.2 Genel (Public) Sayfalar

Bu sayfalar, herhangi bir kimlik doğrulaması gerektirmeksizin tüm ziyaretçilerin erişimine açıktır.

5.2.1 Default.aspx (Ana Sayfa)

Bu sayfanın temel amacı, durumu Published (Yayınlanmış) olarak ayarlanmış quizleri listeleterek ziyaretçilere sunmaktır. Page_Load event'i tetiklendiğinde işleyiş şu adımları takip eder: Veritabanından ilgili quizler çekilir, potansiyel XSS saldırılarına karşı quiz açıklamaları gibi HTML içerikler, Bölüm 4.1.3'te detaylandırılan SanitizeHtml fonksiyonu kullanılarak temizlenir ve son olarak veriler Repeater kontrolü kullanılarak sayfada listelenir.

5.2.2 SolveQuiz.aspx (Quiz Çözme Sayfası)

Bu sayfa, platformun en interaktif bileşenidir ve kullanıcıların quizleri çözdüğü arayüzü barındırır. Sayfa, MultiView kontrolü ile üç aşamalı bir akış yönetir ve kullanıcı cevaplarını Session'da saklayarak durum yönetimini sağlar. Kişilik testi sonuçları, options tablosundaki JSON verisinin parse edilmesiyle hesaplanır.

Quiz Çözme Akışı:

1. **Header View:** Quiz'in başlangıç bilgileri (başlık, açıklama) gösterilir ve kullanıcı "Başla" butonu ile teste geçer.
2. **Questions View:** Sorular, MultiView içinde sırayla gösterilir. Kullanıcının her soru için seçtiği cevap, bir sonraki adıma geçmeden önce Session'da geçici olarak saklanır.
3. **Result View:** Tüm sorular cevaplandıktan sonra, kişilik skoru hesaplama algoritması çalışır ve elde edilen sonuca göre ilgili sonuç tipi (örn: "Aslan") kullanıcıya gösterilir.

Kişilik Skoru Hesaplama Algoritması: Algoritmanın temel mantığı, kullanıcının seçtiği her şıkka karşılık gelen PersonalityScores JSON verisini işlemektir. Her cevap için JSON parse edilir ve içindeki kişilik tiplerine karşılık gelen puanlar, genel bir puan tablosunda toplanır. Quiz sonunda en yüksek puana sahip olan kişilik tipi, nihai sonuç olarak belirlenir.

// Pseudo-code: Puan hesaplama mantığı

```
Dictionary<string, int> personalityScores = new Dictionary<string, int>();  
  
foreach (var answer in userAnswers)  
{  
    string json = GetOptionPersonalityScores(answer.OptionId);  
    var scores = JsonConvert.DeserializeObject<Dictionary<string, int>>(json);  
    foreach (var kvp in scores)  
    {
```

```
if (personalityScores.ContainsKey(kvp.Key))  
    personalityScores[kvp.Key] += kvp.Value;  
else  
    personalityScores[kvp.Key] = kvp.Value;  
}}
```

```
string resultType = personalityScores.OrderByDescending(x => x.Value).First().Key;
```

Submission Kaydı: Quiz tamamlandığında, kullanıcının genel sonucu submissions tablosuna, her bir soruya verdiği cevap ise submissionanswers tablosuna bir veritabanı Transaction'ı içinde kaydedilir.

-- 1. Ana submission kaydı

```
INSERT INTO submissions (QuizId, UserId, ResultId, SubmittedAt) VALUES (@QuizId, @UserId,  
@ResultId, NOW());
```

-- 2. Her cevap için detay kaydı

```
INSERT INTO submissionanswers (SubmissionId, QuestionId, OptionId) VALUES (@SubmissionId,  
@QuestionId, @OptionId);
```

5.3 Kullanıcı (User) Sayfaları

Bu sayfalar, yalnızca giriş yapmış kullanıcıların erişebileceği özel işlevler sunar.

5.3.1 Login.aspx (Giriş Sayfası)

Kullanıcı giriş işlemi, sayfanın code-behind'ından doğrudan bir veritabanı sorgusu ile değil, /api/auth/login endpoint'ine yapılan bir HttpClient çağrısı ile soyutlanmış bir şekilde gerçekleştirilir. İşleyiş şu şekildedir: Kullanıcının girdiği e-posta ve şifre bir JSON nesnesine dönüştürülür, bu nesne API'ye POST isteği ile gönderilir. API'den success yanıtı ve kullanıcı verileri döndüğünde, bu veriler kullanılarak sunucu tarafında Session değişkenleri doldurulur ve kullanıcı ilgili panele yönlendirilir. Eğer hesap durumu inactive ise, kullanıcı hesabını yeniden aktifleştirebileceği ReactivateAccount.aspx sayfasına yönlendirilir.

5.3.2 Register.aspx (Kayıt Sayfası)

Yeni kullanıcı kayıtları, /api/auth/register endpoint'i üzerinden gerçekleştirilir. Bu sayfa aynı zamanda profil resmi yükleme mantığını da içerir.

Profil Resmi Yükleme Mantığı: Kullanıcı bir dosya seçtiğinde, sistem öncelikle bir dizi validasyon uygular: dosya uzantısının izin verilenler listesinde olup olmadığı (.jpg, .png), dosya boyutunun maksimum limiti (5MB) aşıp aşmadığı ve görüntünün piksel boyutlarının belirlenen aralıkta olup olmadığı kontrol edilir. Tüm kontroller başarılıysa, dosya adı çakışmalarını ve path traversal saldırılarını önlemek için GUID ile benzersiz bir dosya adı oluşturulur ve dosya /Uploads/ProfileImages/ klasörüne kaydedilir. Bu süreç, Bölüm 4.1.4'te açıklanan dosya uzantısı, boyut ve path traversal koruması gibi tüm güvenlik validasyonlarını uygular.

5.3.3 Settings.aspx (Ayarlar Sayfası)

Bu sayfa, kullanıcının hesabıyla ilgili üç temel işlemi yapmasına olanak tanır:

1. **Profil Düzenleme:** Kişisel bilgileri /api/auth/update endpoint'ine PUT isteği göndererek günceller.
2. **Şifre Değiştirme:** Eski şifre doğrulaması ile /api/auth/changepassword endpoint'i üzerinden yeni şifre belirler.
3. **Hesap Devre Dışı Bırakma:** /api/auth/deactivate/{userId} endpoint'ine PUT isteği göndererek hesabın IsActive durumunu 0 yapar ve kullanıcıyı sistemden çıkarır.

5.4 Yönetim (Admin) Sayfaları

Bu sayfalar, yalnızca admin rolüne sahip kullanıcılar tarafından erişilebilen, sistem yönetimi araçlarını içerir.

5.4.1 AdminDashboard.aspx (Admin Paneli Ana Sayfası)

Bu dashboard, yöneticilere sistemin genel durumu hakkında hızlı ve anlık istatistikler sunar. Gösterilen her metrik, ilgili tablo üzerinde COUNT(*) sorgusu çalıştırılarak elde edilir:

- **Toplam Quiz Sayısı:** SELECT COUNT(*) FROM quizzes
- **Aktif Kullanıcılar:** SELECT COUNT(*) FROM users WHERE IsActive=1
- **Bekleyen Silme Talepleri:** SELECT COUNT(*) FROM deleterequests WHERE Status='pending'

5.4.2 QuizManagement.aspx (Quiz Yönetimi)

Bu sayfa, sistemdeki tüm quizler üzerinde tam CRUD (Oluşturma, Okuma, Güncelleme, Silme) yetkisi sağlar.

- **Silme İşlemi:** Bir quiz "Sil" butonu ile silindiğinde, veritabanındaki CASCADE DELETE özelliği tetiklenir. Bu sayede, quize bağlı tüm sorular, şıklar, sonuçlar ve kullanıcı çözümleri tek bir DELETE FROM quizzes WHERE Id = @Id sorgusu ile otomatik olarak ve güvenli bir şekilde temizlenir.
- **Status Değiştirme:** Admin, bir quizin durumunu Published ve Draft arasında tek bir tıklama ile değiştirebilir. Bu işlem, ilgili quizin Status alanını güncelleyen basit bir UPDATE sorgusu ile gerçekleştirilir.

5.4.3 QuizCreate.aspx (Quiz Oluşturma Sihirbazı)

QuizCreate.aspx, 2666 satırlık kod tabanıyla projenin şüphesiz en karmaşık ve merkezi bileşenidir. Bu sayfa, yöneticilere 4 adımlı bir sihirbaz aracılığıyla yeni quizler oluşturmaya olanak tanır.

1. **Adım 1: Quiz Bilgileri:** Quiz'in başlığı, CKEditor entegrasyonu ile zengin metin formatında açıklaması ve kapak görseli gibi temel bilgiler bu adımda alınır.
2. **Adım 2: Sonuç Tipleri:** Kişilik testleri için olası sonuç kategorileri (örneğin, "Aslan", "Kaplan") tanımlanır ve bu veriler geçici olarak Session'da bir liste halinde tutulur.
3. **Adım 3: Sorular ve Şıklar:** Bu adım, testin mantığının kurulduğu en kritik bölümdür. Her bir şık için, hangi kişilik tipine kaç puan vereceğini tanımlayan bir PersonalityScores JSON yapısı girilir. Örnek: {"Aslan": 5, "Kaplan": 2, "Kurt": 0}.
4. **Adım 4: Önizleme ve Kaydetme:** "Kaydet" butonuna tıklandığında, Session'da biriktirilen tüm veriler (quiz bilgileri, sonuçlar, sorular ve şıklar) bir veritabanı Transaction'ı içinde sırayla ilgili

tablolara kaydedilir. Bu süreçte herhangi bir hata oluşursa, Rollback işlemi ile tüm değişiklikler geri alınarak veri bütünlüğü korunur.

5.4.4 DeleteRequests.aspx (Silme Talepleri Yönetimi)

Bu panel, yöneticinin kullanıcılar tarafından gönderilen ve durumu pending olan hesap silme taleplerini yönetmesini sağlar.

- **Onaylama İşlemi:** Bir talep onaylandığında, arayüz `/api/admin/deleterequest/{id}` endpoint'ine bir DELETE isteği gönderir. API katmanı, bu isteği alarak veritabanı seviyesinde cascade delete mantığını tetikler ve kullanıcıyla ilişkili tüm verileri (quizleri, çözümleri, profil bilgileri vb.) kalıcı olarak siler.
- **Reddetme İşlemi:** Bir talep reddedildiğinde, arayüz `/api/admin/deleterequest/{id}/reject` endpoint'ine bir PUT isteği gönderir. Bu işlem, talebin durumunu veritabanında rejected olarak günceller.

Tüm bu işlemlerin arkasındaki güç, bir sonraki bölümde detaylandırılan RESTful API katmanından gelmektedir.

6.0 RESTful API Katmanı (Web API)

6.1 API'ye Genel Bakış

Web API katmanı, projenin en temel bileşenlerinden biridir ve Sunum Katmanı (Web Forms) ile Veri Erişim Katmanı arasında bir köprü görevi görür. Bu soyutlama katmanı, ön yüzdeki sayfa mantığının doğrudan veritabanı işlemleriyle ilgilenmesini engelleyerek daha temiz ve yönetilebilir bir mimari sunar. Projedeki tüm 12 endpoint, AuthController sınıfı içinde tanımlanmıştır. API'nin tüm endpoint'leri, parametreleri ve yanıt formatları hakkında detaylı bilgiye, proje çalıştırıldığında `/swagger` adresi üzerinden erişilebilen Swagger dokümantasyonundan ulaşılabilir.

6.2 Endpoint Detayları

API Özet Tablosu

#	Method	Endpoint	Açıklama
1	POST	<code>/api/auth/login</code>	Kullanıcı girişi
2	POST	<code>/api/auth/register</code>	Kullanıcı kaydı
3	PUT	<code>/api/auth/reactivate/{userId}</code>	Hesabı aktifleştir
4	GET	<code>/api/auth/user/{id}</code>	Kullanıcı bilgisi
5	PUT	<code>/api/auth/user/{id}</code>	Profil güncelle
6	PUT	<code>/api/auth/changepassword</code>	Şifre değiştir
7	GET	<code>/api/user/submissions/{userId}</code>	Quiz geçmişi
8	DELETE	<code>/api/admin/deleterequest/{id}</code>	Kullanıcıyı sil
9	PUT	<code>/api/admin/deleterequest/{id}/reject</code>	Talebi reddet

Aşağıda, projedeki en kritik API endpoint'lerinden bazılarının detayları sunulmuştur.

6.2.1 POST `/api/auth/login`

- **Açıklama:** Kullanıcı girişi yapar ve başarılı ise oturumda kullanılacak temel kullanıcı bilgilerini döner.

Method	POST
Content-Type	application/json
Request Body	{ "Email": "user@example.com", "Password": "password123" }

6.2.2 POST /api/auth/register

- **Açıklama:** Yeni bir kullanıcı hesabı oluşturur.

Method	POST
Content-Type	application/json
Request Body	{ "UserName": "john", "Email": "john@example.com", "Password": "pass123", "DisplayName": "John Doe", "Gender": "Erkek", "Birthday": "1990-01-01", "Location": "Ankara", "Bio": "Hello world!", "Interests": "coding, music", "ProfileImage": "default-user.png" }

6.2.3 PUT /api/auth/changepassword

- **Açıklama:** Kullanıcının mevcut şifresini doğrulayarak yenisiyle değiştirir.

Method	PUT
Content-Type	application/json
Request Body	{ "UserId": 1, "OldPassword": "oldpass123", "NewPassword": "newpass456" }

6.2.4 GET /api/user/submissions/{userId}

- **Açıklama:** Belirtilen userId'ye sahip kullanıcının çözdüğü tüm quizlerin geçmişini listeler.

Method	GET
URL Parametresi	userId (int) - Kullanıcı ID
Request Body	Yok

6.2.5 DELETE /api/admin/deleterequest/{requestId}

Method	DELETE
URL Parametresi	requestId (int) - Silme talebi ID
Authorization	Session["UserType"] == "admin"

- **Açıklama:** (Sadece Admin) Bir kullanıcı silme talebini onaylar ve kullanıcıyla ilişkili tüm verileri kalıcı olarak siler. Quiz silme gibi daha basit senaryolarda veritabanının CASCADE DELETE özelliği yeterliyken, burada kullanıcının tüm ayak izini (kendi oluşturduğu içerikler dahil) kontrollü bir şekilde silmek için API katmanında, bir transaction içinde yönetilen sıralı bir silme mantığı tercih edilmiştir.
- **Yetkilendirme:** Bu endpoint'e erişim, sunucu tarafında Session["UserType"] == "admin" kontrolü ile korunmaktadır.
- **Silme Sırası:** Veritabanındaki foreign key kısıtlamaları nedeniyle silme işleminin belirli bir sırada yapılması zorunludur. API bu sırayı şu şekilde yönetir:
 1. SubmissionAnswers
 2. Submissions
 3. Options
 4. Questions
 5. Results
 6. Quizzes
 7. DeleteRequest
 8. User

Bu yapı, bir geliştiricinin projeyi kendi makinesinde nasıl çalıştıracağını açıklayan son bölüme zemin hazırlar.

7.0 Kurulum ve Başlatma Kılavuzu

7.1 Geliştirme Ortamı Gereksinimleri

Projeyi yerel geliştirme ortamınızda sorunsuz bir şekilde derlemek ve çalıştırmak için aşağıdaki yazılım ve araçların yüklü olması gerekmektedir:

- Visual Studio 2022 (veya 2019)
- .NET Framework 4.7.2
- MySQL 8.0+ (MySQL Server ve MySQL Workbench)
- IIS Express (Visual Studio ile birlikte gelir)

7.2 Kurulum Adımları

Projeyi sıfırdan kurup çalıştırmak için aşağıdaki adımları izleyin:

1. **Veritabanı Kurulumu:**
 - MySQL Workbench uygulamasını açın.

- Proje dosyaları içinde bulunan web_quiz_platform_db.sql dosyasını içe aktararak (import) veritabanı şemasını ve başlangıç verilerini oluşturun.
- Proje kök dizinindeki Web.config dosyasını açın ve connectionStrings bölümündeki bağlantı dizesini kendi MySQL sunucu bilgilerinize (sunucu adresi, kullanıcı adı, şifre) göre güncelleyin.

2. Proje Kurulumu:

- Proje reposunu GitHub'dan yerel makinenize klonlayın.
- Visual Studio'da *.sln uzantılı solution dosyasını açın.
- Solution Explorer penceresinde solution'a sağ tıklayıp "Restore NuGet Packages" seçeneğini seçerek projenin bağımlılıklarını yükleyin.

3. İzinler:

- Proje klasöründeki /Uploads/ klasörüne, web sunucusunun (IIS Express) yazma izni (write permission) olduğundan emin olun. Bu, kullanıcıların profil resmi veya quiz görselleri yükleyebilmesi için gereklidir.

4. Çalıştırma:

- Tüm adımlar tamamlandıktan sonra, projeyi başlatmak için Visual Studio'da F5 tuşuna basmanız veya "Start Debugging" butonuna tıklamanız yeterlidir. Proje, IIS Express üzerinde çalışmaya başlayacak ve varsayılan tarayıcınızda açılacaktır.

8. Yapayzeka Kullanımı

Projede yapayzeka, frontend geliştirme sürecinde yardımcı araç olarak kullanılmıştır.

8.1. QuizCreate.aspx Wizard JavaScript

- Step navigation logic (goToStep fonksiyonu)
- Dynamic form validation
- CKEditor integration
- Dosya upload preview sistemi
- JSON entry ve validation
- PersonalityScores input sistemi

8.2. Admin Panel UI

- Dashboard kartlarının responsive düzeni
- GridView filtreleme ve pagination
- Modal confirmation dialogs
- Color scheme ve ikon seçimi
- Responsive navbar tasarımı

Not: API, veritabanı ve DbHelper kendi bilgi ve araştırmalarım ile geliştirilmiştir. Yapayzeka backend, frontend JavaScript ve UI tasarımı için yardımcı araç olarak kullanılmıştır.

9. Kurulum Talimatları

9.1. Gereksinimler

- Visual Studio 2022 (veya 2019)
- MySQL 8.0+
- IIS Express
- .NET Framework 4.7.2

9.2. Database Kurulumu

1. MySQL Workbench açın
2. web_quiz_platform_db.sql import edin
3. Web.config connection string güncelleyin

9.3. Proje Kurulumu

1. GitHub'dan klonlayın
2. Visual Studio'da .sln açın
3. NuGet paketlerini restore edin
4. Uploads klasörüne write permission verin
5. F5 ile çalıştırın

10. Proje Tanıtım Videosu

Video Linki:

https://drive.google.com/drive/folders/1eLbGl8x_wDEdTDaIK4PM2SPmXPPSoQwI?usp=drive_link

Kod yapısı anlatımı (API, DbHelper, ASPX)

- Veritabanı şeması
- Canlı demo
- Admin panel özellikleri
- Yapay zeka kullanımı

11. Sonuç ve Değerlendirme

Bu proje, ASP.NET Web Forms ve Web API teknolojileri kullanılarak modern bir quiz platformu geliştirmiştir. 9 RESTful API endpoint, 8 tablolu veritabanı, cascade delete mekanizması, session-based authentication, file upload sistemi ve admin paneli ile kapsamlı bir web uygulaması oluşturulmuştur.

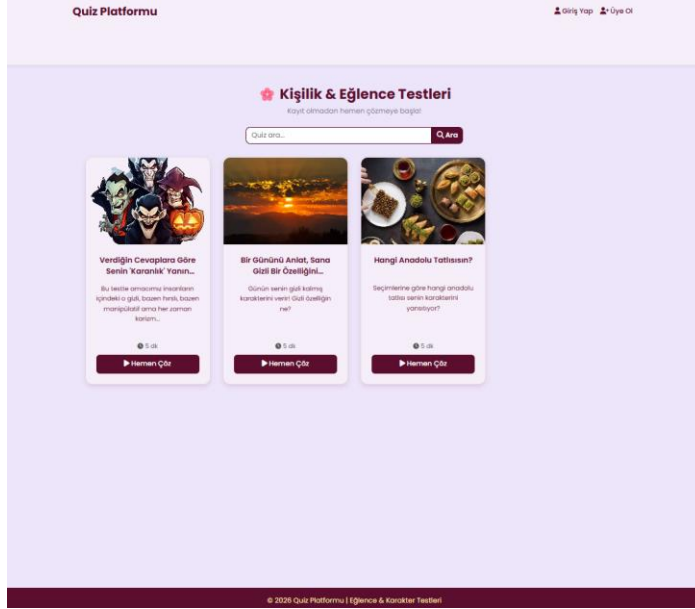
11.1. Kazanılan Beceriler

- ASP.NET Web Forms ve Web API hibrit kullanımı
- RESTful API tasarımı (12 endpoint)
- MySQL database tasarımı ve ADO.NET
- Session-based authentication
- File upload ve validation
- Frontend-backend entegrasyonu
- Yapay zeka destekli geliştirme

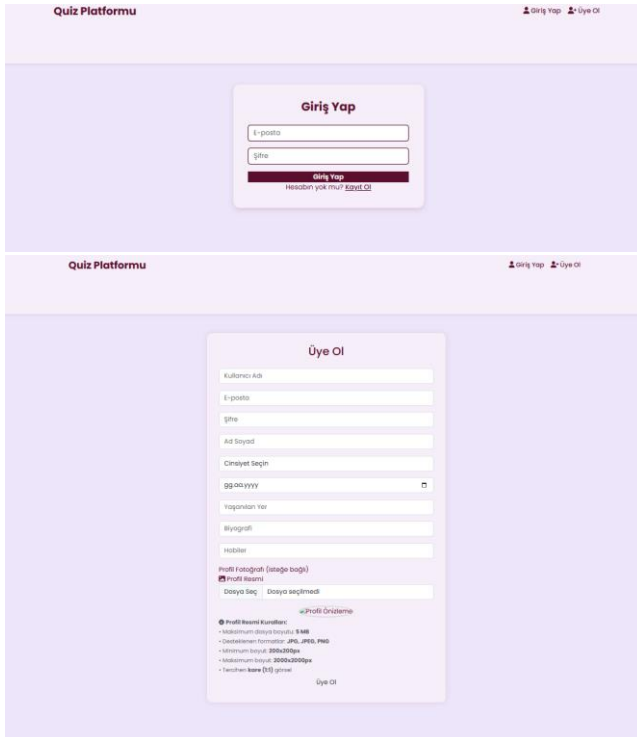
--- RAPOR SONU ---

Görsel Arayüzler

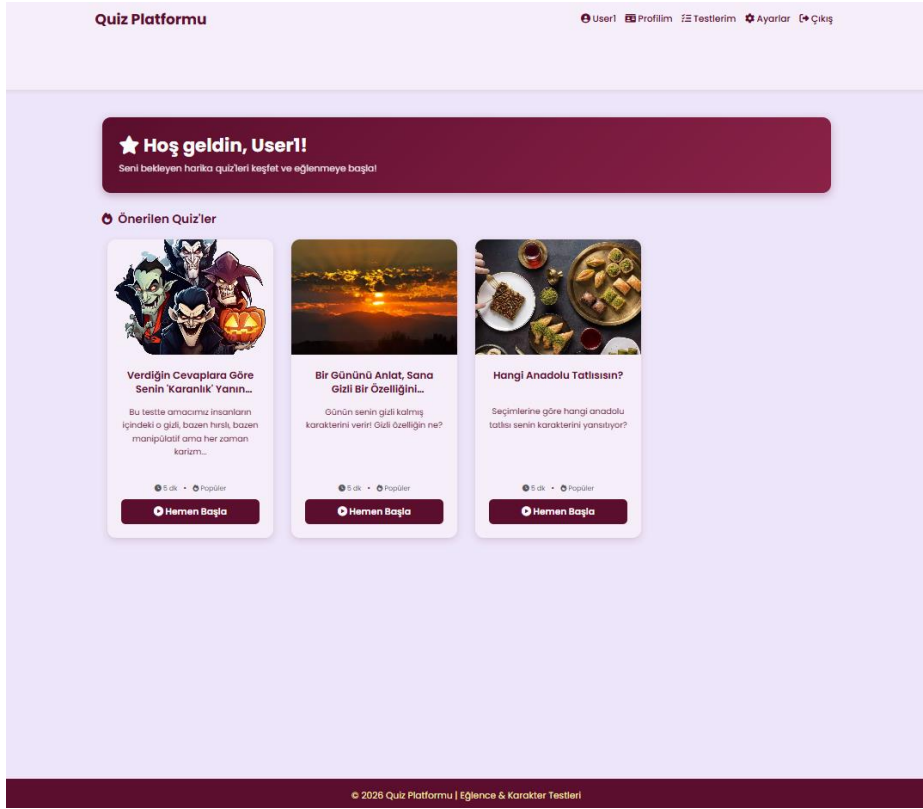
Görsel 1. Default.aspx sayfası



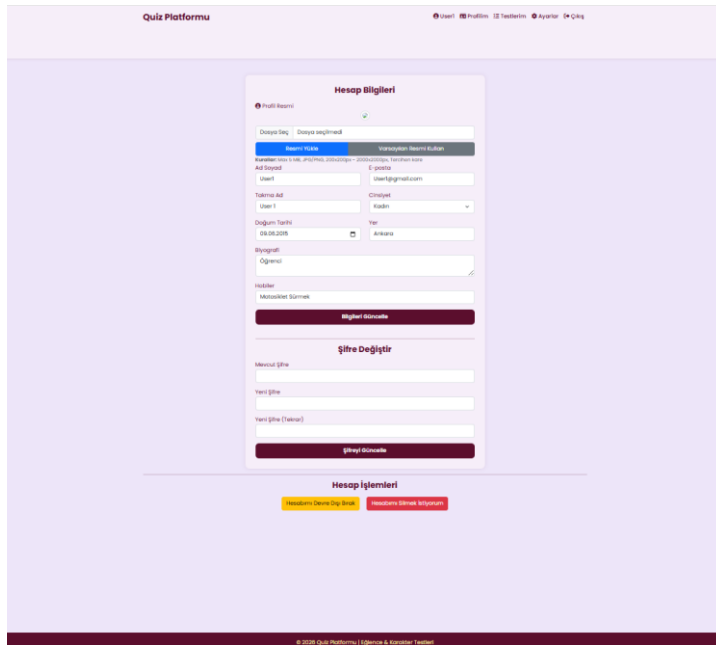
Görsel 2. Ve 3. Login.aspx ve Register.aspx sayfaları



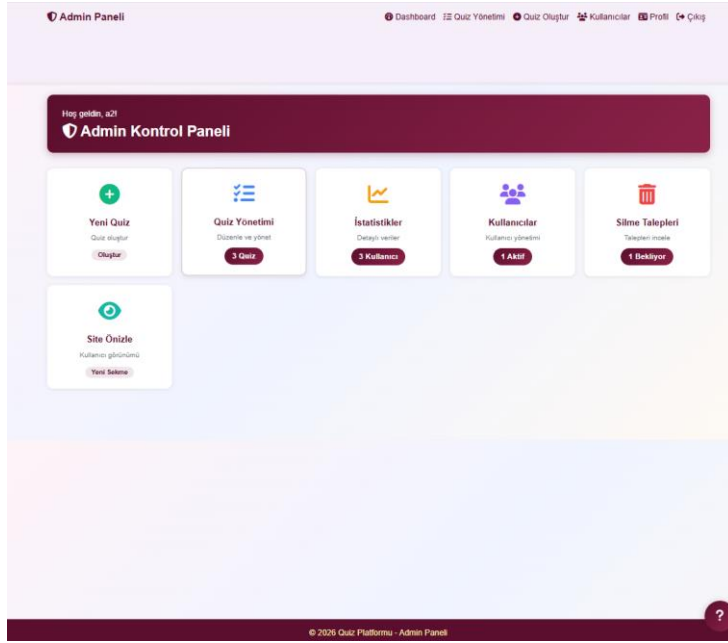
Görsel 4. UserDashboard.aspx Sayfası



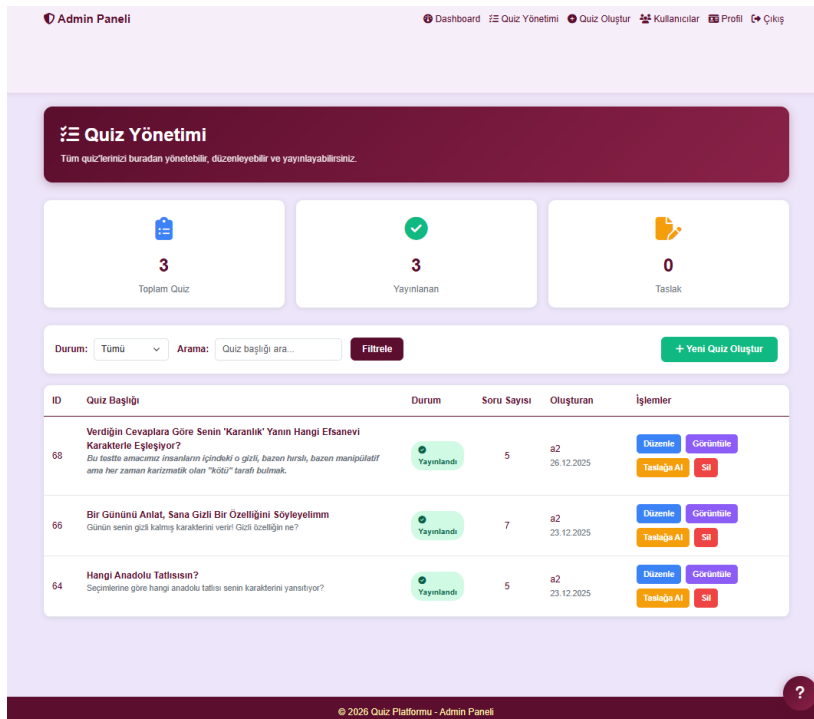
Görsel 5. Settings.aspx Sayfası



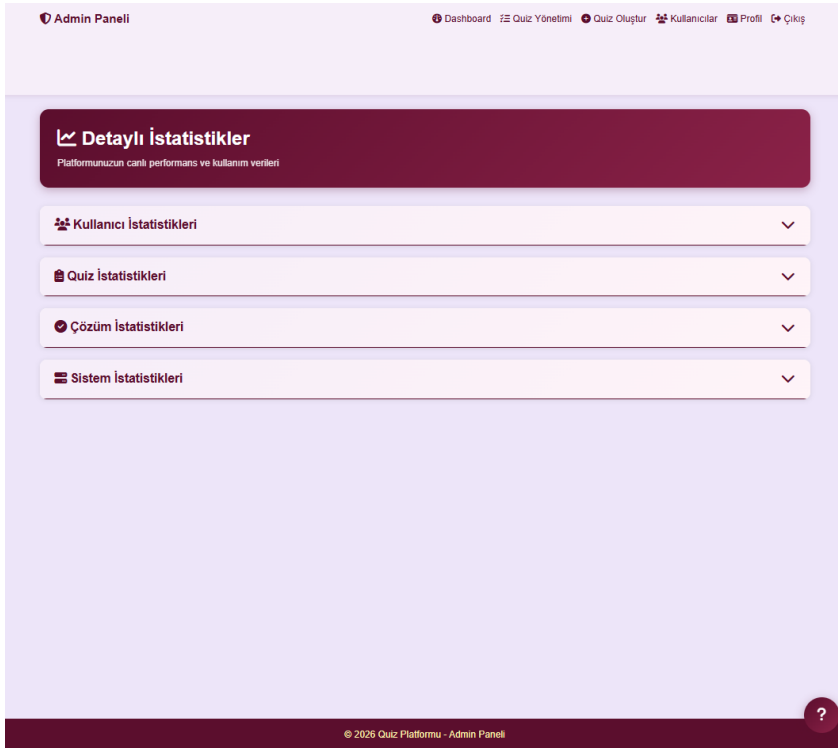
Görsel 6. AdminDashboard.aspx Sayfası



Görsel 7. QuizManagement.aspx Sayfası



Görsel 8. Statistics.aspx Sayfası



Görsel 9. UserManagement.aspx Sayfası

Admin Paneli

Dashboard Quiz Yönetimi Quiz Oluştur Kullanıcılar Profil Çıkış

Kullanıcı Listesi

Kullanıcı adı veya e-posta ara...

Ara Temizle

Kullanıcı Adı	E-posta	Rol	Aktif Mi?	Durum	Rol	Sil
user3	user3@gmail.com	user	Evet	Aktif/Pasif	Rol Değiştir	Sil
user2	user2@gmail.com	user	Evet	Aktif/Pasif	Rol Değiştir	Sil
User1	User1@gmail.com	user	Evet	Aktif/Pasif	Rol Değiştir	Sil
a2	a2@gmail.com	admin	Evet	Aktif/Pasif	Rol Değiştir	Sil

© 2026 Quiz Platformu - Admin Paneli